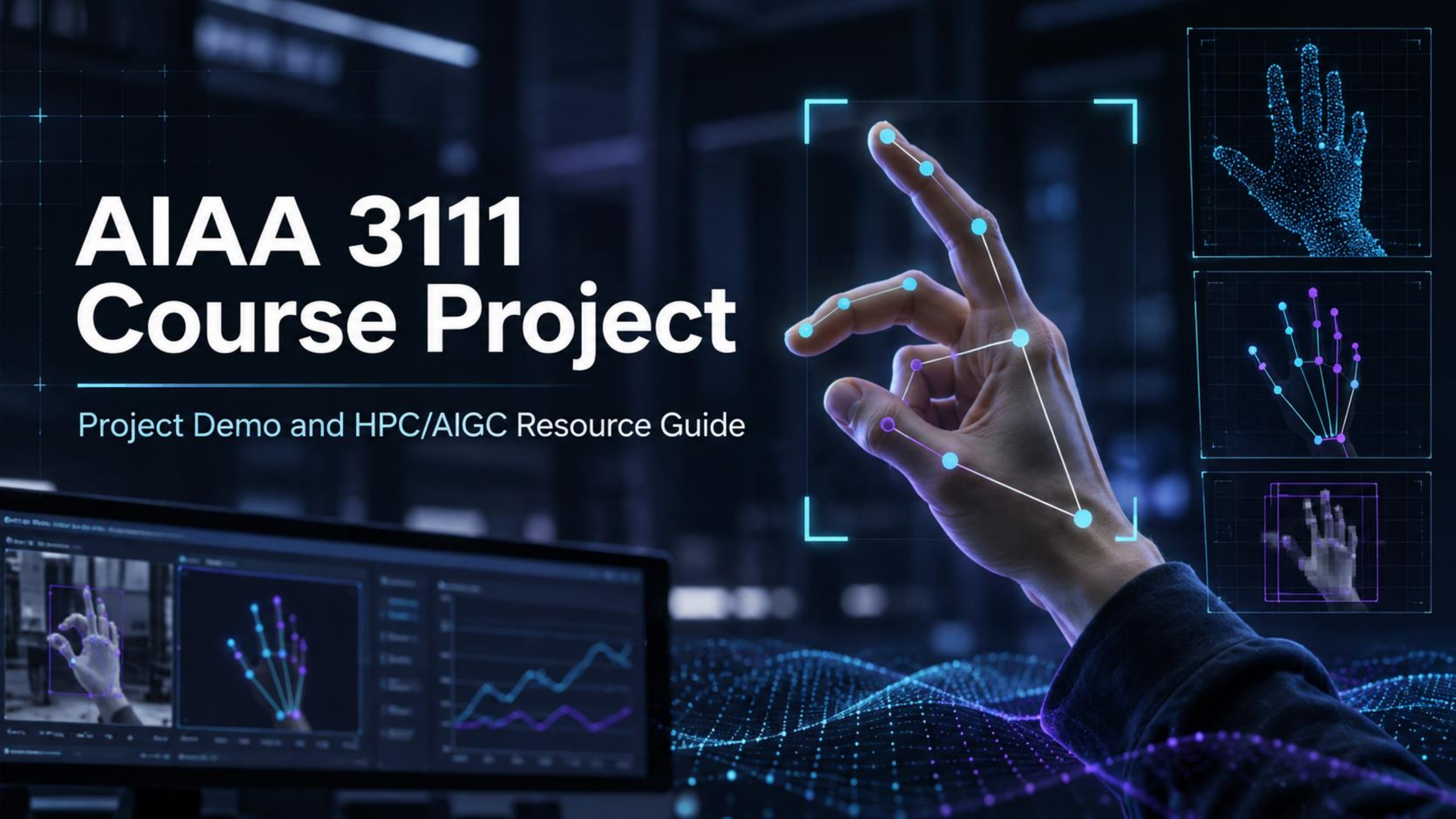
AIAA 3111 Course Project

Project Demo and HPC/AIGC Resource Guide



Today's Session

1. Project Requirements
2. Gesture Recognition Mini-Project
3. HPC and AIGC Resources
4. Student Checklist



Project Deliverables and Schedule



15 OCT · 23:59

TEAM + PROPOSAL

Form a team of 2–4 on Canvas

Submit title, brief introduction and member names

Proposal review through Oct 22

27 NOV · 23:59

POSTER SUBMISSION

Submit **Poster.pdf** separately

Poster guidance will be released later

04 DEC · 15:00–16:20

POSTER SESSION

Present the project and answer questions

11 DEC · 23:59

FINAL SUBMISSION

Submit **Group_XX_Project.zip**

18 DEC · 23:59

PEER EVALUATION

Individual and confidential

FINAL PACKAGE: Report.pdf · code · README · data or download instructions

Poster.pdf is not required in the final ZIP

LATE POLICY:
submissions up to one week late lose **20%** of the earned mark

Project Topic Options

Choose at least one data mining task

CLASSIFICATION

Robust Human Mobility Classification



CLUSTERING

Social Media Community and Topic Evolution



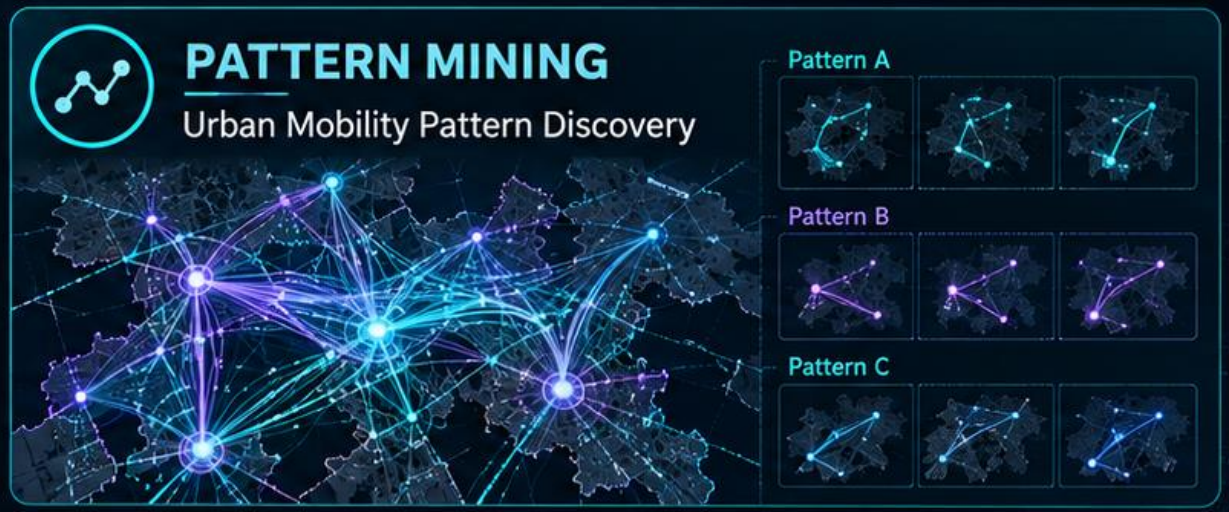
ANOMALY DETECTION

Spatio-Temporal Urban Sensor Anomaly Detection



PATTERN MINING

Urban Mobility Pattern Discovery

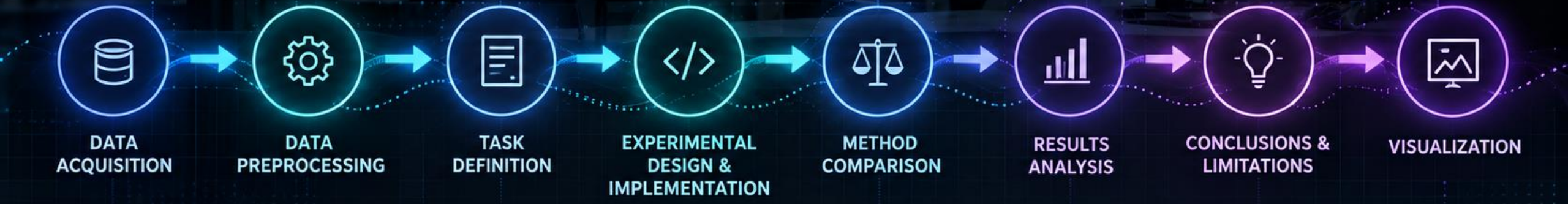


Reference examples are optional. You may adapt an example or propose your own topic.

What the Project Should Include

Every topic must follow a complete data mining workflow

Do not submit directly AI-generated work or fabricated references. Please describe how AIGC was used.



REPORT STRUCTURE

Introduction → Problem Formulation → Methodology → Experiments → Conclusion and Limitations → Disclosures

6–10 pages of main text

What the Poster Should Show

Tell a clear, standalone project story

1 PROBLEM & MOTIVATION



Real-world need and why it matters.



2 METHODOLOGY

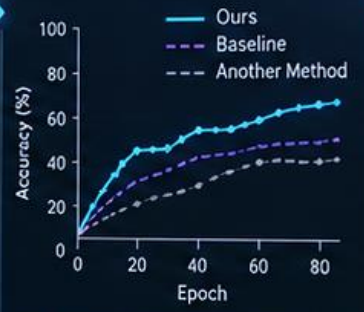
How you approach the problem.



Preprocessing → Model Design → Training → Evaluation

3 KEY RESULTS

Quantitative and qualitative evidence.

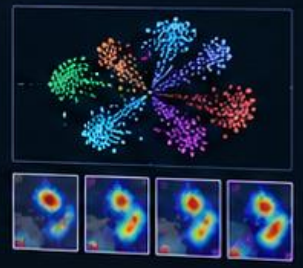




4 MAIN FINDINGS

What the results mean.

- Outperforms baselines
- More robust across conditions
- Learns meaningful representations



5 PRACTICAL IMPLICATIONS

Potential real-world impact.



Safer and smarter systems



Supports real-world deployment



Applicable to related domains

6 LIMITATIONS

What doesn't work yet or could be improved.



Challenging conditions (e.g., low light, occlusion)



Limited performance on rare/long-tail cases



Sensitive to domain shift and noisy inputs

7 CLEAR FIGURES & READABLE LAYOUT

Use clean, well-labeled figures and a logical flow.



The audience should understand the main project **without reading the full report.**

Use the poster as a visual anchor **during your presentation** and Q&A.

Poster Example 1 — Sleep Monitoring System

WHAT THEY DID

A non-contact sleep monitor using mmWave radar to estimate breathing and heartbeat.

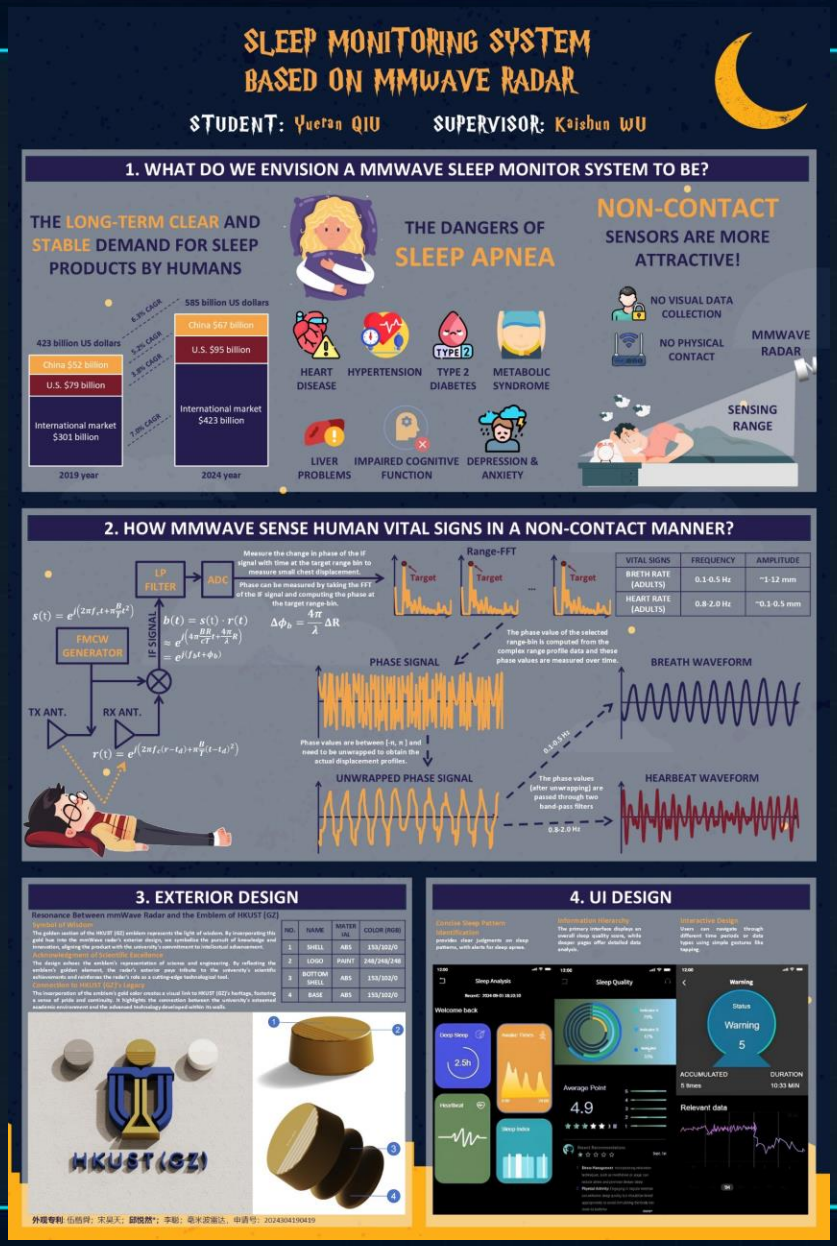
WHAT WORKS

- Clear healthcare motivation
- Complete sensing-to-interface concept
- Strong diagrams and visual hierarchy

WHAT TO IMPROVE

- No participants or dataset details
- No ground truth or evaluation protocol
- No quantitative accuracy or error
- No baseline comparison or limitations

Takeaway: A complete system still needs experimental validation.



Poster Example 2 — Biometric Authentication

WHAT THEY DID

Mobile typing patterns are used to verify whether two inputs come from the same user.

WHAT WORKS

- Clear task, data, and features
- Multiple models and metrics
- Failure analysis improves the design

WHAT TO IMPROVE

- Too much text
- Key findings are buried
- Results need visual charts
- Limitations are not explicit

Takeaway: Strong analysis needs clear visual communication.

Supervised Learning Methods for Biometric Authentication on Mobile Devices

Valerie Ding[1], Stephanie Dong[2], Jonathan Li[3]
Department of Computer Science, Stanford University
[1]dingv@stanford.edu, [2]sxdong11@stanford.edu, [3]johnnyli@stanford.edu

Abstract

- We develop **logistic regression** models and **deep neural networks** to classify mobile device users based on biometric typing pattern data.
- We present a secure, space-efficient, extensible framework for **real-time biometric fraud detection** on mobile devices.

Introduction

- Keystroke pattern and dynamics classification is an important application of machine learning to computer security and authentication. The massive increase in popularity and **computing power of mobile devices** in the last ten years has spurred significant interest in biometric authentication models for mobile devices.
- Existing literature emphasizes need for more nuanced **security protocols** in personal devices. As mobile devices store increasingly valuable and confidential information, learning classifiers to detect fraud is becoming ever more applicable and important.
- At the same time, a general, space-efficient, and real-time framework is required to be viable in practice. To this end, we develop fraud detection algorithms that use real-time keystroke dynamics data, and propose a **space-efficient real-time authentication framework** that can be integrated into native software across all mobile devices.

Data and Features

- The MEU-Mobile KSD (Keystroke Dynamics) Data Set from the UCI Machine Learning Repository contains 51 records for each of 56 subjects - 2856 records total - of haptic, momentum, and timing features measured of a common sequence (.tie5Roanl) typed on a Nexus 7 mobile device. There are 71 features monitored, characterized by the attributes Hold, Up-Down, Down-Down, Pressure, Finger-Area, Average Hold, Average Pressure, Average Area.
- We trained a variety of binary classifiers to detect if the concatenation the feature vectors of two data-points, forming a vector 142 features, was typed by the same user or not. This allowed us to train one model and have it generalize user verification to any new users not from the training dataset, as long as we one at least one keystroke record for any new users.
- As part of data processing we implemented a flexible **resampling framework** that can utilize a variety of undersampling and oversampling methods to undersample the majority class and oversample the minority class as necessary. This ensures parity between labels of different user and same user in the training data.

71 features

	Subject	Hold .	...	AvA
56 subjects 51 records/ subject	1	89	...	0.2880184
	...	...	...	...
	56	80	...	0.260369

Table 1. Feature vector format.

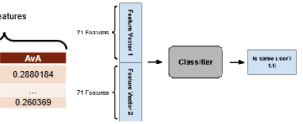


Figure 1. User Verification Classifier.

Models and Discussion

- **Logistic regression with cross entropy loss and no resampling**
  For the first 10 examples each of the first 10 users, with 70%-30% train-validation split, we achieved **89.6% accuracy** with 0% precision, 0% recall, and 100% specificity. Upon inspection, the model consistently predicted the 0 label for every single validation example. We hypothesized that the disproportionate prediction of label 0 was due to heavily unbalanced data, with a significant majority class 0.

- **Logistic regression with cross entropy loss and 50-50 undersampling**
  In our next attempt, we used resampling techniques to balance the majority and minority class to parity. With 50-50 undersampling using random undersampling of the majority class, and using our entire post-processed dataset of 8 million comparative examples, we achieved 50.0% accuracy, 50.0% precision, 100% recall, and 00.02% specificity. This means 50% likelihood of predicting same user when the user was in fact different. This is not more effective than a random guess, so the challenge will be lowering the false positive count, as it is more important, from a security point of view, to minimize false positives (predict same user, but actually different) than false negatives (predict different user, but actually same).

$$\text{IsSameUser}(x) = [\sigma(Wxb) > 0.5]$$

- **Fully Connected Deep NN with cross entropy loss and 50-50 undersampling**
  From the results in the section above, we hypothesized a single logistic unit could not represent enough complexity to capture the relationship between the 142 features of our input. Hence, we trained a variety of fully connect deep neural networks and compared their validation accuracy. Deep neural nets ranging from 1 hidden layer to 5 hidden layers, with 10 neurons per hidden layer, with ReLU activation, and sigmoid activation on the output layer. The loss function remained cross entropy. We trained each DNN model for 25 epochs from randomly initialized weights and measured their validation accuracy. This was repeated 10 times for each DNN model, and the average of validation accuracy of 10 trials was recorded as a benchmark of how each additional layer improve the performance of the model.

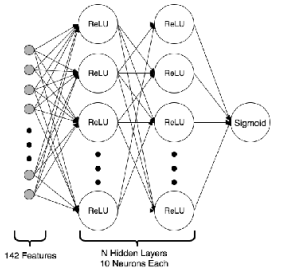


Figure 2. Deep neural network training architecture. We use ReLU activation for hidden layers (10 units each) and sigmoid for output. We experimented on varying number of hidden layers.

Results

- From this investigation, we conclude with 10 ReLU activated neurons per hidden layer, the 3 hidden layer model achieves the best accuracy vs. training speed trade-off. Hence, we retrained that model for 100 epochs, and produced the following validation results: **79.8% accuracy, 81.7% precision, 76.8% recall, and 95.1% specificity.**

Model	Loss	Accuracy	Recall	Precision	False Negative Rate
Logistic	6.661	58.12%	23.88%	78.18%	76.12%
DeepNN 1	0.690	69.10%	49.05%	**83.31%**	50.95%
DeepNN 2	7.905	50.41%	**99.78%**	49.76%	**0.22%**
DeepNN 3	0.536	73.73%	70.37%	76.04%	29.63%
DeepNN 4	0.695	49.73%	0.21%	59.89%	99.79%
DeepNN 5	0.531	73.45%	72.52%	74.30%	27.48%
DeepNN 6	0.409	81.70%	91.18%	77.40%	8.82%
DeepNN 7	0.484	76.88%	97.16%	69.15%	2.84%
DeepNN 8	0.527	72.93%	80.23%	70.79%	19.77%
DeepNN 9	0.425	80.29%	81.69%	80.19%	18.31%
DeepNN 10	0.450	79.78%	77.46%	81.37%	22.54%
Triangle	**0.380**	**84.28%**	89.76%	81.14%	10.24%

Table 2. Deep Neural Net models and their average validation accuracy of 10 training trials

Conclusions and Future Work

- We developed logistic regression and deep neural network classifiers for user verification through biometric typing pattern data on mobile devices, achieving **79.8% accuracy**. This means that 79.8% of the time, our classifier was able to successfully differentiate between two user inputs.
- Applications of these discriminatory classifiers are in enhancing device security by adding another **layer of verification**. By writing this classifier onto mobile devices and training it on a user's featurized password input, we can ensure that even if a password's content is typed in properly, it must be typed in with the learned cadence of the original user in order to be verified. This will effectively proof every mobile device from brute force password attacks by adding an unknown number of additional features the attacker must account for. Additionally, this has the ability to continually verify the authenticity of the user based on their typing patterns as they use the mobile device, hardening against device takeover by ensuring that only the primary user has access to the phone.
- **Future work:** In this research, we resampled by downsampling the majority class. Our flexible resampling framework allows for different resampling techniques, which can be explored in the future. Additionally, we can perform data augmentation using adversarial examples or other upsampling techniques. This would help inform next-generation development of discriminatory classifiers.

How the Project Is Assessed

100 points across five components and 11 criteria

20	**Peer Assessment**	Contribution · Teamwork
20	**Poster**	Content · Visual Communication
30	**Presentation / Q&A**	Project Understanding • Answers and Evidence • Interpretation
20	**Report**	Problem and Methods • Experiments and Analysis
10	**Code**	Functionality · Readability and Setup

The highest-weight component is the **live presentation and Q&A.**

https://github.com/b-rbmp/cv-gesture-recognition

PUBLIC CASE STUDY

Real-Time Gesture Recognition

From image classification to a mobile Rock–Paper–Scissors game

PROBLEM → DATA → COMPARISON → ANALYSIS → DEPLOYMENT

Why this case: it shows the complete path from a defined problem to a tested application.



Sapienza Computer Vision 2023–2024 project · Used as an attributed teaching example

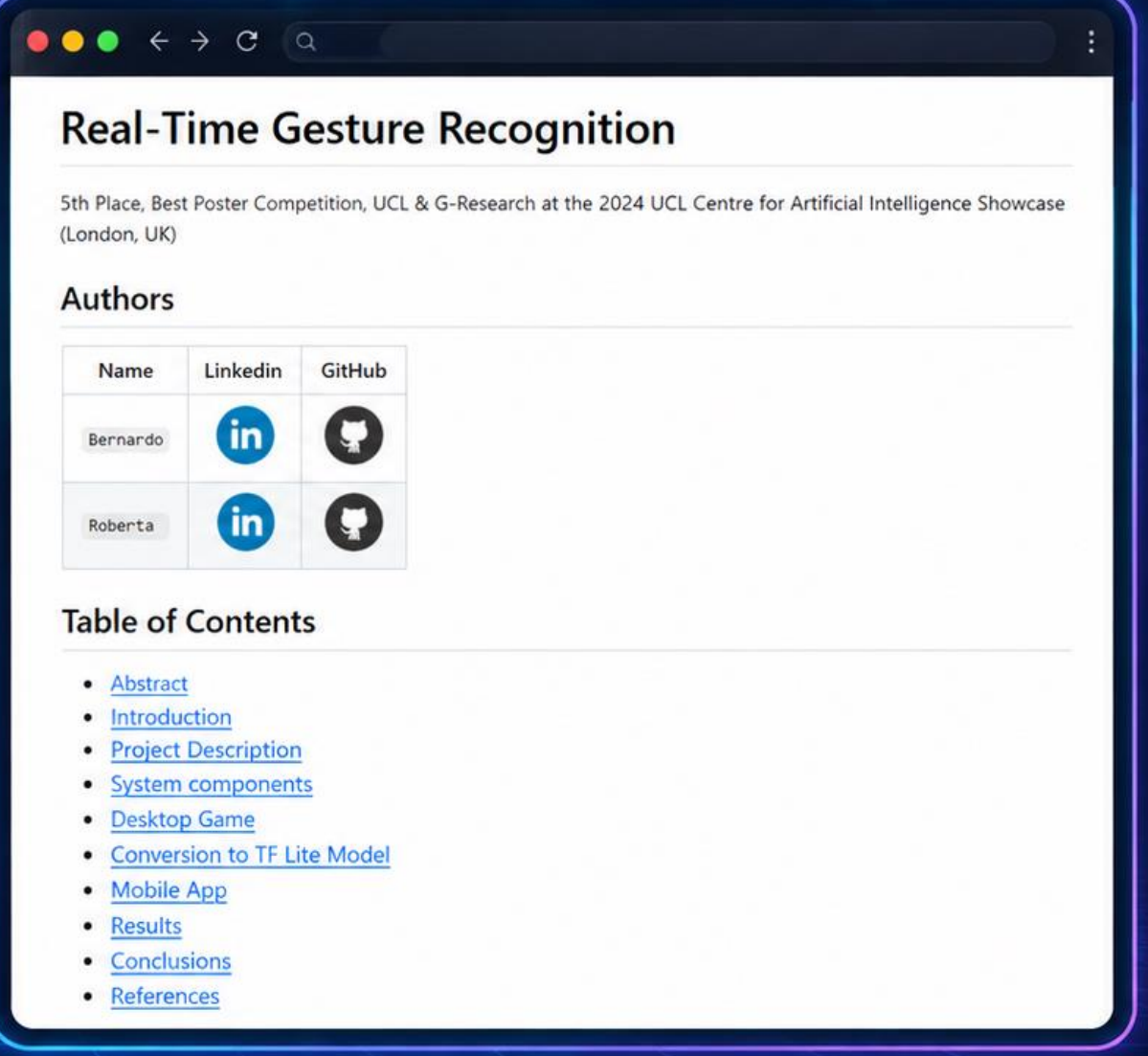
Explore the Full Project Repository

WHAT YOU CAN FIND

- Project motivation and objectives
- Dataset, preprocessing, and model code
- Experiments, metrics, and result figures
- Desktop game, mobile app, and TF Lite conversion

» Use the repository to inspect the full workflow and implementation details.

README TABLE OF CONTENTS



Real-Time Gesture Recognition

5th Place, Best Poster Competition, UCL & G-Research at the 2024 UCL Centre for Artificial Intelligence Showcase (London, UK)

Authors

Name	Linkedin	GitHub
Bernardo		
Roberta		

Table of Contents

- Abstract
- Introduction
- Project Description
- System components
- Desktop Game
- Conversion to TF Lite Model
- Mobile App
- Results
- Conclusions
- References

https://github.com/b-rbmp/cv-gesture-recognition

Motivation

Natural and Contact-Free Interaction



PROJECT DEMO

Mobile Rock-Paper-Scissors

Hand gesture recognition can provide **intuitive control** when a keyboard, mouse, or touchscreen is inconvenient.

Problem Formulation

19-Class Image Classification

OBJECTIVE Recognize a static hand gesture and use the prediction to control an application

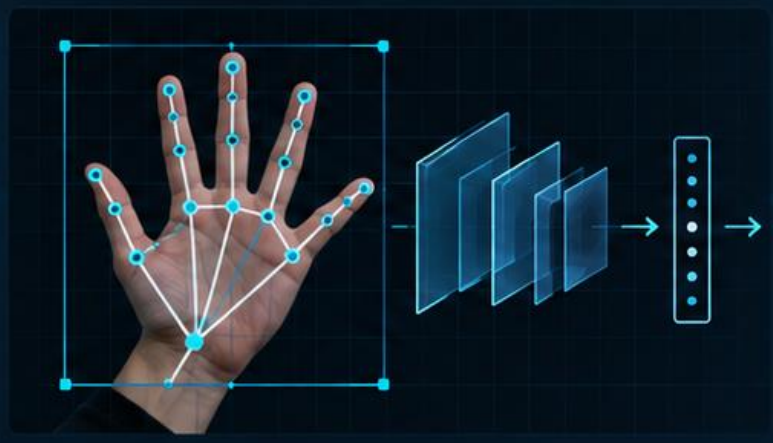
INPUT

One RGB camera frame



TASK

Classify one of 19 gesture classes



OUTPUT

One predicted gesture class



EVALUATION

Accuracy · Precision · Recall · F1 score

DEPLOYMENT CONSTRAINT

The final model must run in real time on a smartphone

https://huggingface.co/datasets/cj-mills/hagrid-classification-512p-no-gesture-150k-zip

Methodology

Data preparation and model configurations

DATASET

Adapted HaGRID classification dataset

153,735 images · 19 classes · about 6,900 images per class



PREPROCESSING

- Train / validation / test split
- Resize to 300 × 300 · normalize to [0, 1]



AUGMENTATION

- Flip · rotation · brightness · saturation · sharpen · blur · crop
- 15% probability for each transformation

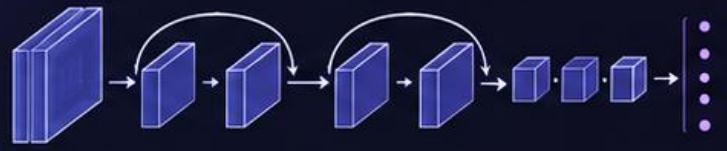


MODELS

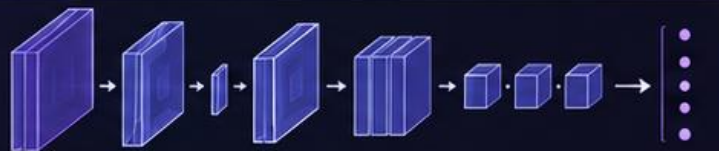
1. Simple CNN



2. ResNet50



3. MobileNetV3 Large



4. Frozen HandSeg + MobileNetV3



5. Unfrozen HandSeg + MobileNetV3



Experiments

Three stages answer three different questions



1 MODEL COMPARISON

Compare all five architectures

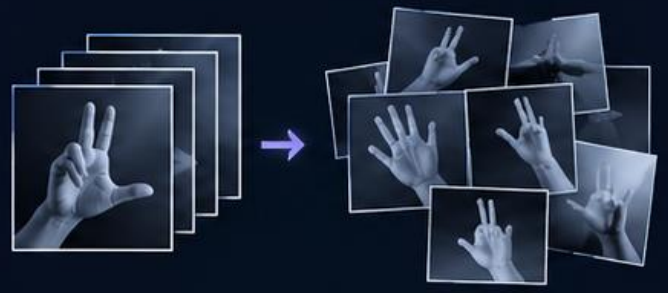


20% of original data for training / validation
10% for testing · no augmentation

Question: Which architecture performs best under the same conditions?

2 AUGMENTATION ABLATION

Compare ResNet50 with and without data augmentation

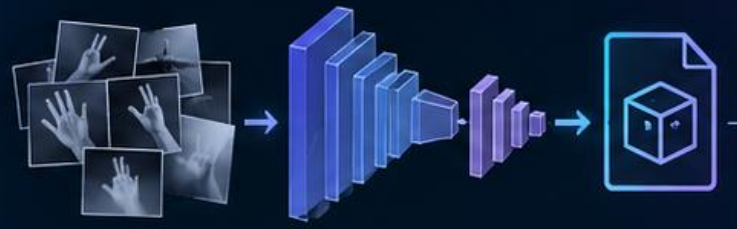


Question: Does augmentation improve performance?

3 FULL-DATA TRAINING

Train MobileNetV3 Large on the full augmented dataset

Convert the final model to TensorFlow Lite



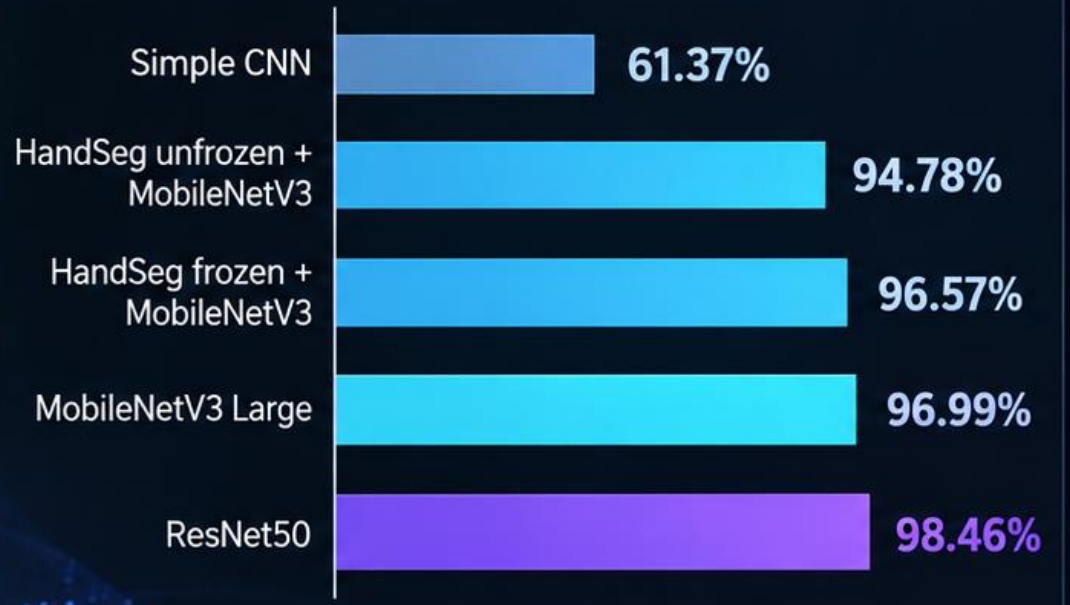
Question: Can the selected model support the mobile application?

FAIR COMPARISON: keep the data, evaluation protocol and metrics consistent

Analysis

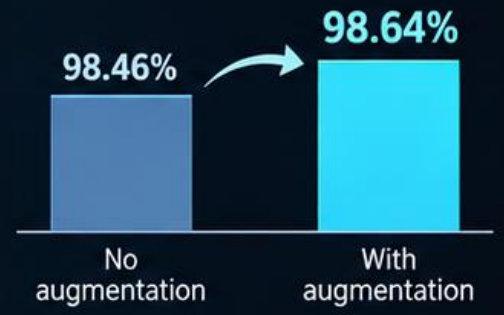
Accuracy alone does not determine the final model

SHARED SUBSET • NO AUGMENTATION



AUGMENTATION

ResNet50:
98.46% → 98.64%

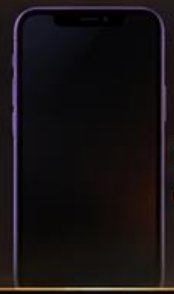


+0.18 percentage points

DEPLOYMENT TRADE-OFF

ResNet50:
highest subset accuracy · failed to load on the phone

98.46%



Failed to load on the phone

MobileNetV3:
slightly lower subset accuracy · deployed successfully

96.99%



Deployed successfully

MODEL SELECTION MUST MATCH THE COMPLETE OBJECTIVE

Conclusion

MobileNetV3 Large provides the best **balance** for the complete objective

FULL AUGMENTED DATASET



MobileNetV3 Large



TensorFlow Lite



MOBILE APPLICATION



Successfully tested on a Google Pixel 8

98.87%

Accuracy · Precision · Recall · F1 score

FINAL SELECTION

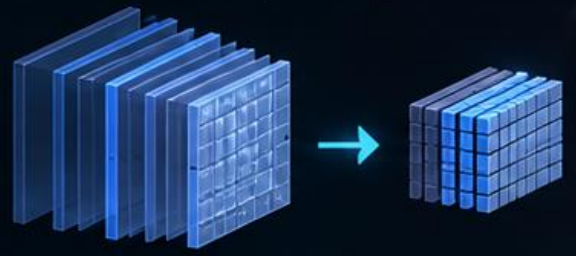
Strong classification performance and successful mobile deployment

Limitations and Future Work

A successful demo still has clear boundaries

MODEL OPTIMIZATION

Reduce memory use with quantization or pruning



REAL-TIME PERFORMANCE

Improve speed and test performance across devices



USER EXPERIENCE

Add gesture feedback, customization and multiplayer support



GESTURE COVERAGE

Expand beyond the limited game gestures



Limitations are not weaknesses to hide. They define the next questions to test.

Why This Counts as a Complete Project

1
Motivation
gesture interaction



2
Problem
19-class classification

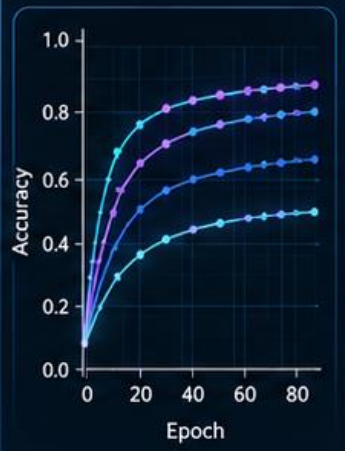


3
Methodology
preprocessing and five models

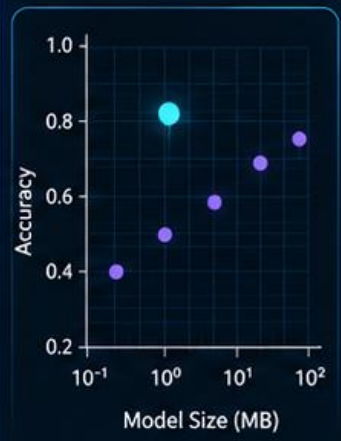


CNN
ResNet
DenseNet
EfficientNet
MobileNetV3

4
Experiments
controlled comparisons



5
Analysis
accuracy and deployment trade-off

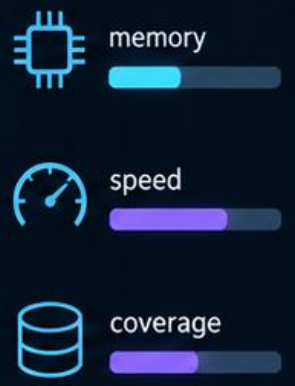


6
Conclusion
MobileNetV3 selected



MobileNetV3

7
Limitations
memory, speed, coverage



GPU and HPC Workflow



1. Campus Wi-Fi or VPN



2. Connect to HPC



3. Prepare environment



4. Transfer code and data



5. Request GPU



6. Run job



7. Inspect logs and results





50 GPU hours per student

GPU Access Documentation URL: https://docs.hpc.hkust-gz.edu.cn/docs/hpc12/connection

AIGC API Setup and Responsible Use

Python

```python
import os
from openai import OpenAI

api_key = os.getenv("AIGC_API_KEY")
if not api_key:
    raise RuntimeError("Please set AIGC_API_KEY "
                       "in your environment.")

client = OpenAI(
    api_key=api_key,
    base_url="https://aigc-api.hkust-gz.edu.cn/v1",
)

response = client.chat.completions.create(
    model="DeepSeek-V4-Flash",
    messages=[{"role": "user", "content": "Hello!"}],
)
print(response.choices[0].message.content)
```



Protect the key

Keep your API key in environment variables.
Do not share, commit, or expose it.



Control token cost

Be concise, avoid unnecessary context,
and set reasonable limits (e.g. max tokens).



Verify every output

Check facts, logic, code, and citations.
Do not rely on AI output without review.



Disclose AIGC use

Clearly acknowledge when you use AIGC
in your assignments or projects.



RMB 50 API credit per student

API Documentation URL: https://docs.hpc.hkust-gz.edu.cn/docs/aigc/api-usage



群聊：AIAA3111 (L02)

该二维码7天内（10月17日前）有效

Team Formation Timeline

COURSE WECHAT GROUP

Open to all students for project discussion and team matching

1 — OCT 11

JOIN THE GROUP

Join the course WeChat group

2 — OCT 11

COMPLETE THE FORM

Share a project idea or indicate your interests

3 — OCT 11 EVENING

IDEAS SHARED

Submitted project ideas will be shared in the group

4 — BY OCT 12, 23:59

SELF-MATCHING

Form a team of 3–4 students and register on Canvas

5 — OCT 13 AFTERNOON

ASSISTED ASSIGNMENT

Remaining students will be matched by interest where possible

Random assignment will only be used when preference information is unavailable.

Project Start Checklist

1. Form a team
2. Choose a real-world problem
3. Design comparisons before coding
4. Build a reproducible workflow



- 01. Group Project Notice.pdf
- 02. Project_Assessment_Rubrics.pdf
- 03. Project.pdf
- 04. Canvas Team Formation Guide.pdf
- 05. Report Template.docx
- 06. AIGC_API_USER_GUIDE.pdf

Questions?